



Aksum University
Faculty Of Computing Technology
Department Of Computer Science
Report On Real Time And Embedded System
Individual Assignment

FreeRTOS-Based LED Control System on ATmega32

Course: Embedded Systems

Platform: ATmega32 | Microchip Studio | Proteus 8

Submitted By: Haftye Hluf
Id Aku1601850

Submitted To: Zeberhe Guesh (Ms.c)

Table of Contents

1. Introduction	3
2. System Design and Hardware Connections	3
3. FreeRTOS Integration	3
4. Task Creation and Priority Assignment	4
5. Observed LED Behavior	5
6. Challenges Faced.....	5
1. Insufficient Heap Size.....	5
2. Risk of Stack Overflow.....	5
3. Clock Configuration Accuracy	6
4. Missing Dynamic Allocation Support	6
7. Simulation Results.....	6
8. Conclusion.....	6
9. Referance	7

1. Introduction

This project presents the implementation of a real-time operating system (RTOS) on an 8-bit AVR microcontroller. FreeRTOS is utilized to manage three concurrent tasks, each responsible for controlling an LED with a different blinking frequency. The main objective is to gain practical understanding of task creation, priority-based scheduling, and preemptive multitasking using FreeRTOS on the ATmega32 microcontroller.

2. System Design and Hardware Connections

The hardware setup was designed and simulated using Proteus 8 Professional. The ATmega32 microcontroller operates at a frequency of 8 MHz using an external crystal oscillator along with two load capacitors.

Three LEDs are connected to PORTB of the ATmega32 as shown below:

LED Color MCU Pin Resistor Connection

Green	PB0	330Ω	→ GND
Yellow	PB1	330Ω	→ GND
Red	PB2	330Ω	→ GND

Each LED is connected in series with a 330Ω resistor to limit current and protect both the LED and the microcontroller pins. The pins PB0, PB1, and PB2 are configured as output pins using the DDRB register in the software.

Hardware Initialization Code:

```
void hardware_init(void)
{
    DDRB  |= (1 << GREEN_LED_PIN) | (1 << YELLOW_LED_PIN) | (1 <<
RED_LED_PIN);
    PORTB &= ~( (1 << GREEN_LED_PIN) | (1 << YELLOW_LED_PIN) | (1 <<
RED_LED_PIN) );
}
```

3. FreeRTOS Integration

FreeRTOS was integrated into the Microchip Studio project by manually adding the required source files:

- `tasks.c` — Handles core task management
- `list.c` — Implements internal linked lists used by the scheduler

- `queue.c` — Provides queue and semaphore functionality
- `port.c` — AVR-specific port layer (handles context switching and tick interrupts)
- `heap_1.c` — Basic memory allocation scheme
- `FreeRTOSConfig.h` — Contains project-specific configuration settings

Key Configuration Settings in `FreeRTOSConfig.h`:

```
configUSE_PREEMPTION           = 1           // Enables preemptive scheduling
configCPU_CLOCK_HZ             = 8000000     // 8 MHz system clock
configTICK_RATE_HZ             = 1000        // 1 ms tick period
configMAX_PRIORITIES           = 4           // Priority levels from 0 to 3
configMINIMAL_STACK_SIZE       = 85          // Stack size per task (in words)
configTOTAL_HEAP_SIZE          = 1500        // Total heap memory in bytes
configUSE_16_BIT_TICKS         = 1           // 16-bit tick counter for AVR
configSUPPORT_DYNAMIC_ALLOCATION = 1          // Enables dynamic task creation
```

A tick rate of 1000 Hz means the scheduler executes every 1 ms. Therefore, `pdMS_TO_TICKS(200)` directly corresponds to 200 ticks, maintaining a 1:1 conversion.

4. Task Creation and Priority Assignment

Three tasks are created inside the `main()` function using `xTaskCreate()` before starting the scheduler:

```
xTaskCreate(vGreenLEDTask, "GreenLED", TASK_STACK_SIZE, NULL, 3, NULL);
xTaskCreate(vYellowLEDTask, "YellowLED", TASK_STACK_SIZE, NULL, 2, NULL);
xTaskCreate(vRedLEDTask, "RedLED", TASK_STACK_SIZE, NULL, 1, NULL);
```

```
vTaskStartScheduler();
```

Task Name	LED	Priority	Blink Rate
<code>vGreenLEDTask</code>	Green (PB0)	3 (High)	200ms ON / 200ms OFF
<code>vYellowLEDTask</code>	Yellow (PB1)	2 (Mid)	500ms ON / 500ms OFF
<code>vRedLEDTask</code>	Red (PB2)	1 (Low)	1000ms ON / 1000ms OFF

Each task follows a similar infinite loop structure: turning the LED ON, delaying, turning it OFF, and delaying again.

```
void vGreenLEDTask(void *pvParameters)
{
    (void)pvParameters;
    for(;;)
    {
        PORTB |= (1 << GREEN_LED_PIN);
        vTaskDelay(pdMS_TO_TICKS(200));
        PORTB &= ~(1 << GREEN_LED_PIN);
        vTaskDelay(pdMS_TO_TICKS(200));
    }
}
```

```
}  
}
```

The function `vTaskDelay()` is essential because it places the task into the **BLOCKED** state for a specified duration, allowing other tasks to execute. This enables multiple tasks to run effectively on a single-core microcontroller.

5. Observed LED Behavior

After the scheduler starts, all three LEDs operate independently and appear to blink simultaneously:

- **Green LED:** fastest blinking rate — completes a full cycle every 400 ms
- **Yellow LED:** moderate speed — completes a full cycle every 1000 ms
- **Red LED:** slowest — completes a full cycle every 2000 ms

Since tasks spend most of their time in the **BLOCKED** state (due to `vTaskDelay()`), CPU contention is minimal. However, when multiple tasks become **READY** at the same time (e.g., at 1000 ms), the scheduler executes them based on priority: Green first, then Yellow, then Red. The switching delay is extremely short (microseconds), making all LEDs appear to blink concurrently.

Task States:

- **RUNNING** → Task is currently executing
- **READY** → Task is prepared to run but waiting for CPU access
- **BLOCKED** → Task is delayed or waiting for an event

The FreeRTOS tick interrupt occurs every 1 ms and checks if any blocked task has completed its delay period. If so, the task transitions from **BLOCKED** to **READY**, and the scheduler selects the highest-priority **READY** task for execution.

6. Challenges Faced

1. Insufficient Heap Size :Initially, `configTOTAL_HEAP_SIZE` was set too low, causing the scheduler to fail without clear errors. Each task requires memory for its Task Control Block (TCB) and stack. With three tasks using 85 words each, plus kernel overhead, 1500 bytes was found to be the minimum stable configuration for the ATmega32's 2KB RAM.

2. Risk of Stack Overflow

The ATmega32 has only 2048 bytes of RAM. Allocating too large a stack per task would quickly exhaust available memory. A size of 85 words (170 bytes) per task was adequate for these simple tasks, which do not use large local variables or deep call stacks.

3. Clock Configuration Accuracy

The value of `configCPU_CLOCK_HZ` must exactly match the actual hardware clock frequency. Any mismatch leads to incorrect timing behavior. Setting it to 8000000 ensured correct operation with the 8 MHz crystal in the Proteus simulation.

4. Missing Dynamic Allocation Support

Some FreeRTOS versions require explicitly enabling dynamic memory allocation. Defining `#define configSUPPORT_DYNAMIC_ALLOCATION 1` resolved compilation issues related to `xTaskCreate()`.

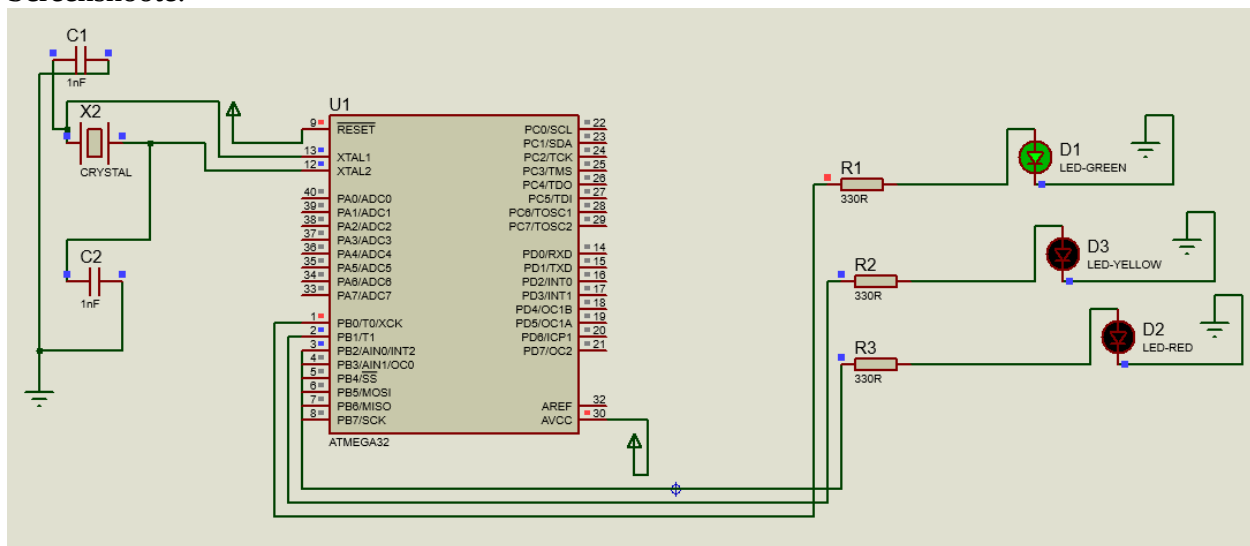
7. Simulation Results

The system was tested using Proteus 8 Professional. The simulation verified the following:

- ✚ All three LEDs blink correctly at their assigned rates
- ✚ The system operates with low CPU utilization (~5%), indicating efficient scheduling
- ✚ The circuit functions properly with the external crystal and resistors
- ✚ No runtime or simulation errors were observed

The Proteus environment displayed an “ANIMATING” status with approximately 5% CPU load, confirming that FreeRTOS is managing task switching efficiently without overloading the microcontroller.

Screenshoote:



8. Conclusion

This project successfully demonstrated multitasking using FreeRTOS on the ATmega32 microcontroller. Three independent LED control tasks were implemented with different priorities and timing intervals, all running concurrently. The use of `vTaskDelay()` allowed cooperative

multitasking by efficiently sharing CPU time among tasks. FreeRTOS simplifies embedded system design by enabling developers to write each task independently while the scheduler manages execution and timing. The Proteus simulation validated the expected behavior, confirming correct implementation and system performance.

9. Referance

1. **FreeRTOS**
FreeRTOS Kernel Developer Documentation.
Available at: FreeRTOS Official Documentation
2. **Microchip Technology**
ATmega32 Datasheet.
Available at: ATmega32 Datasheet
3. **Microchip Studio**
Microchip Studio for AVR Development.
Available at: [Download Microchip Studio](#)